

Monday	4	11	18	25	M
Tuesday	5	12	19	26	A
Wednesday	6	13	20	27	R
Thursday	7	14	21	28	2
Friday	1	8	15	22	0
Saturday	2	9	16	23	2
Sunday	3	10	17	24	4
Week	9	10	11	12	13

FEB 2024

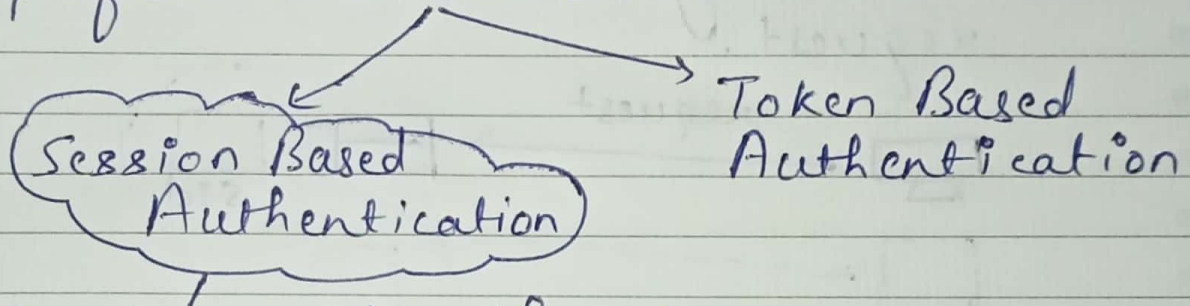
monday

(050-316) Week 08

19

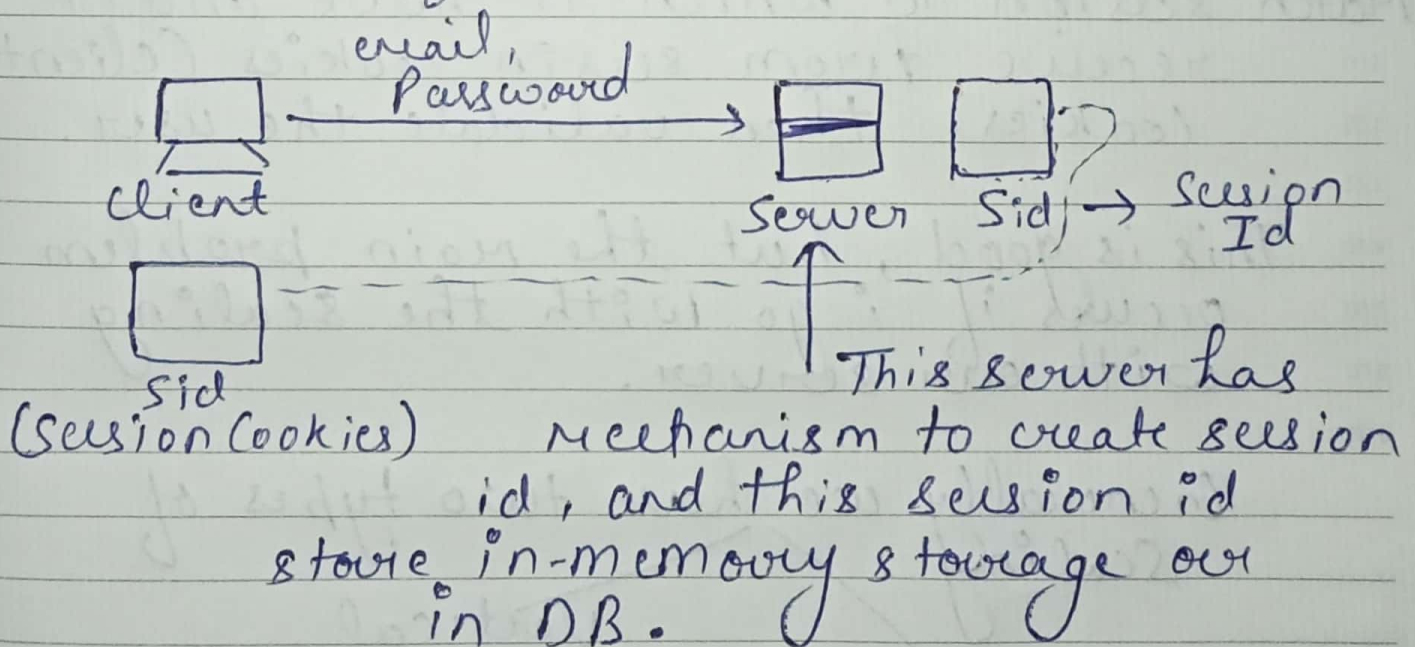
Authentican

Generally we have two famous ways to perform Authentication.



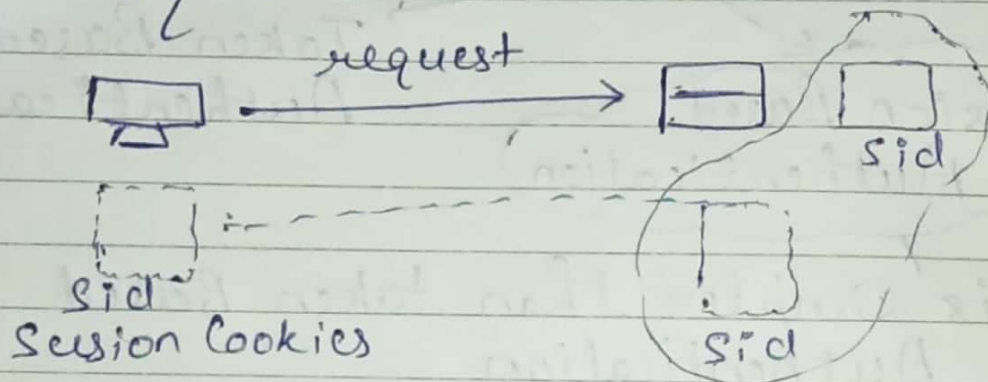
This is simpler than token Based Authentication

In terms of Web-based Application



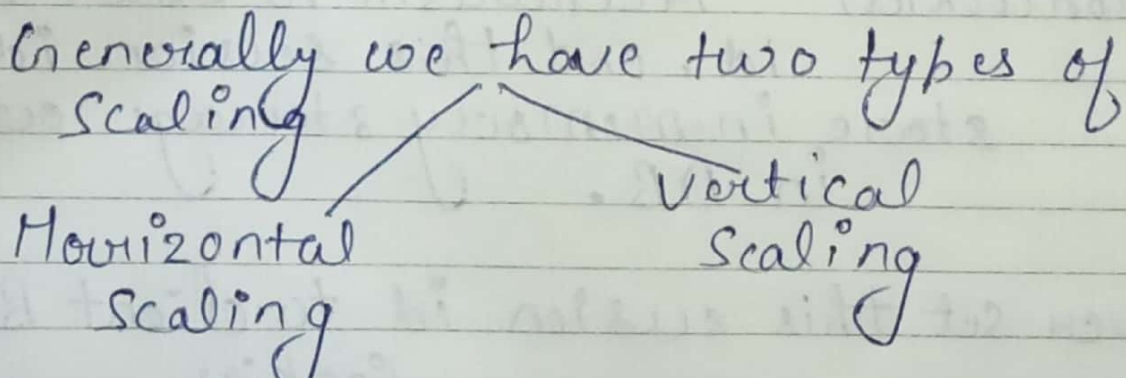
Server set this session id to client Browser Cookies.

And, we know if anything store in cookies, so whenever we request something to server session cookies will also automatically send to server with request.



Here server has mechanism to check our match session id which it have and it receive from session cookies (client cookies) then validate the user.

This is good, but the main problem occurs if i go with the scaling with our server.



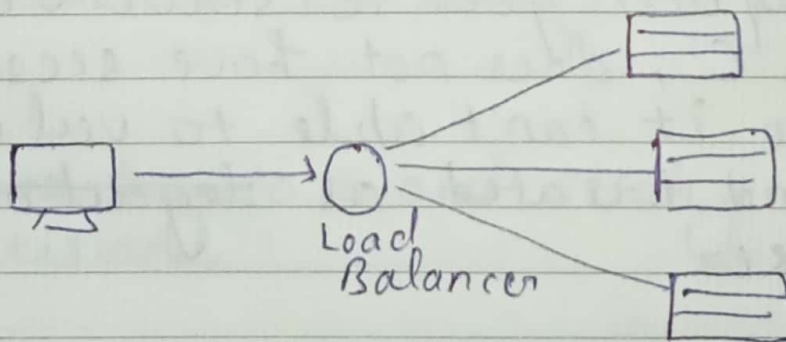
Vertical scaling → session Based

Authentication is good work in vertical scaling,
But every device has its own capacity to scale.

Horizontal → Every company uses

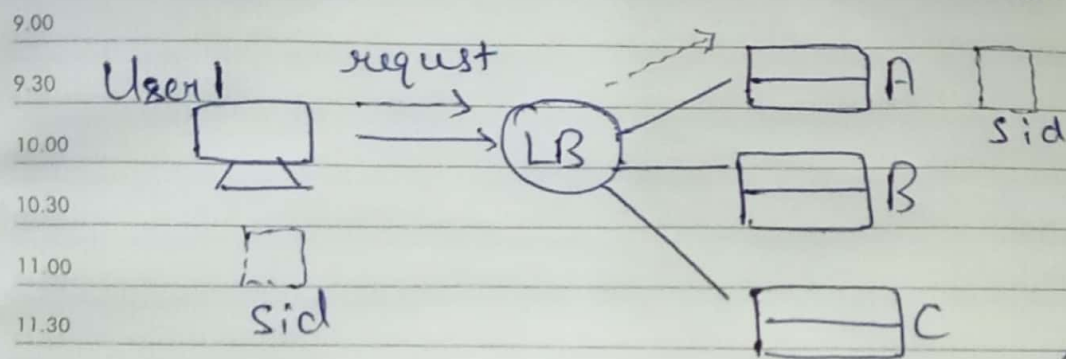
Horizontal scaling with the help of Load Balancer.

Means creating multiple servers and manage n. no of client request via Load Balancer.



This LB follow round Robin Algorithm to divide equally the client request.

Suppose User1, request to login on server A.



If request go to server A, so server A have sid, only A Server can validate, not others becoz server A have only User1 sid

So only server A have sees in id of User1, so according to round robin if next time User1 request redirect to server B. or server C so it can't be able to validate User1 becoz they don't have sid although they treated User1 as new user or logged out user.

This server B, C again create sid for same User1. Becoz of this User need to provide his credentials to login, But as per session it was not expired, but still need to login as a logout User.

Sticky session

If UI requests server A for login, so server A have session id of UI.

So, if we don't follow RRA Algo and send every request of UI to server A.

Stick to that server which have its session id.



Load Balancer is come to equally handle client request But this violates the LB rule.

So we try to avoid sticky sessions.

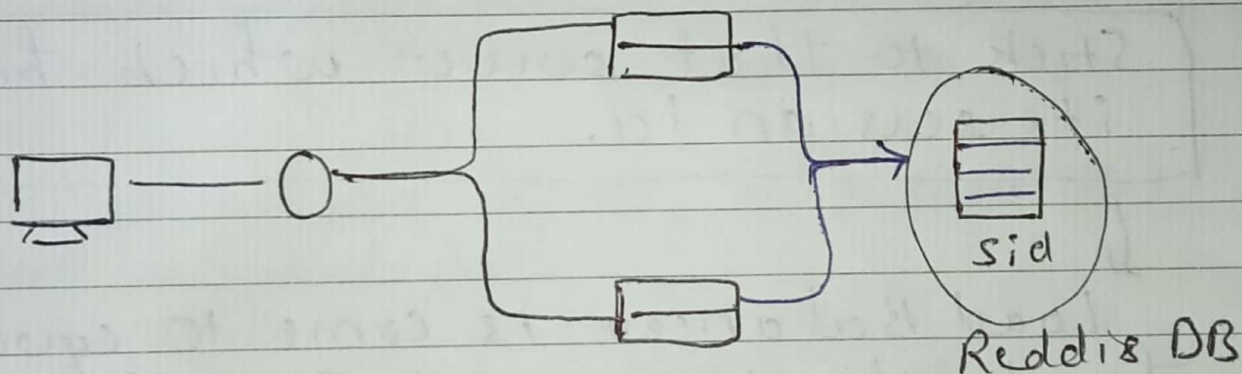
sunday
(028-338) Week 04

28

There is one more option which is better than sticky session →

Storage of session in Centralised location

- In memory storage
- Any database like MongoDB & SQL
- Use very fast DB is Redis



Now every server store session id at centralised location.

Whenever user send it cookies for validation, server find session id in DB.

Although there is also a problem DB show single point of failure.

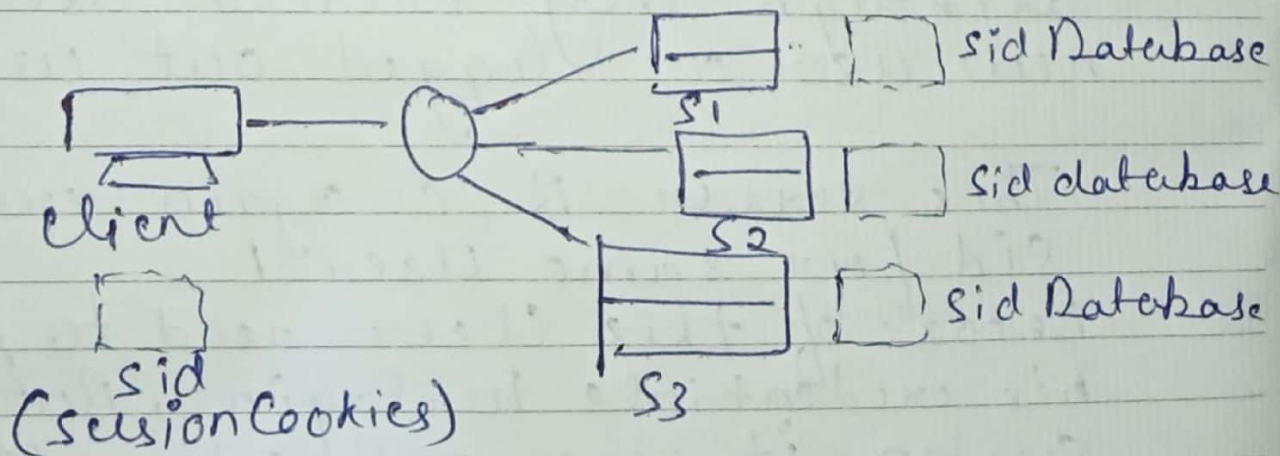
If in any case DB Corrupt, so that Session Based Authentication is failed.

There is one way to resolve Above issue -

Data Grids - Replicate all session id with all server.

But there is also one more problem may be all server our session id are not properly synchronized with each other, or facing some delays

Because of this, it may cause an Data Inconsistency Issue.



There is two technical term

Statefull
↓

Session Based Authentication is an example of Stateful Web

Stateless
↓

Token Based Authentication is an example of stateless web.

1) Remember client data

1) Does not remember client data

2) Store state & client information

2) Does not store state & client information

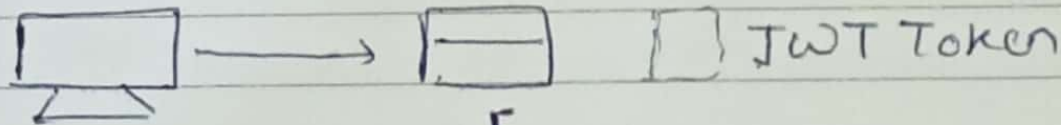
3) Overhead, not simple

3) Simple & Robust

Rest API means stateless Request which means request A & request B are in isolated, not know each other.

To avoid Session Based Authentication
 we implement token Based Authentication.

Token Based Authentication (stateless)



No this server create a
 JWT token and server cookies ke
 andar push krke bhi return krskta
 hai, and our response me return
 kia jata hai.

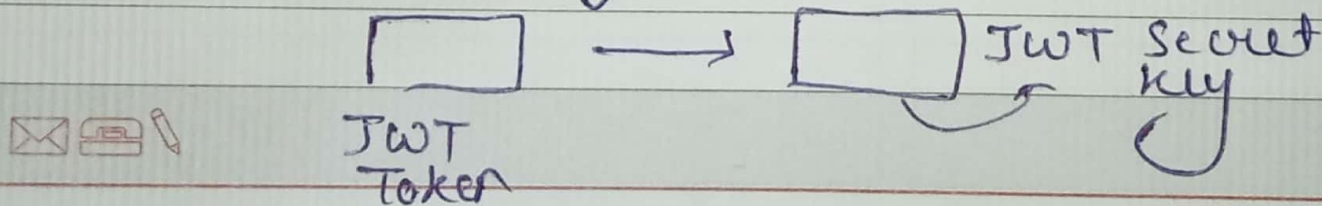
And this token store in Cookies,
 local storage & session.

1.30 • But if JWT token store in cookie,
2.00 so with every request cookies get token
2.30 transfer hoga. And server just
3.00 need to validate.

3.30
4.00 • If store in session, this is temporary
4.30 storage.

5.00
5.30 Now in token Based Authentication
6.00 token is not stored at server side,
But it store at client browser,

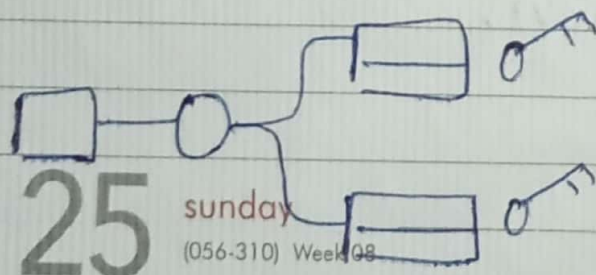
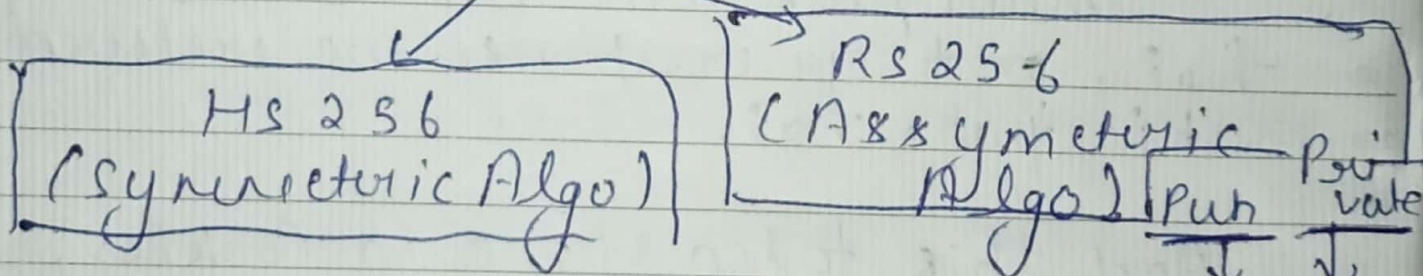
How server validate JWT Token
Secret key



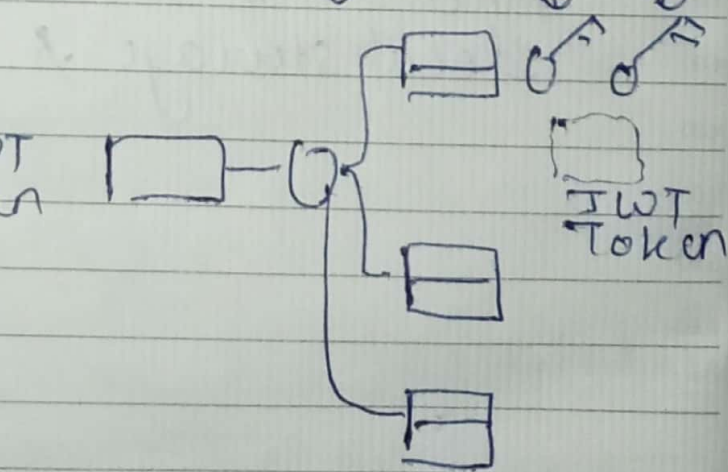
This token Based Authentication also may ~~be~~ cause 2 problems :-

- 1) Make sure to secure our JWT Secret from server.
- 2) Make sure secret key from every server are in sync mode

To resolve this issue we have two Algorithm



Sign & verify JWT Token with the same key



Here token sign with private key and validate by public key.

Session Based Authentication is simpler than token based Authentication.

RS256 is the solution of RS256

• In, HS256, we need to make sure at every server my secret create key is secure our not.

• If i change any secret key, so we have to change on every server.

• Here no ~~su~~ issue like we have to make sure sync our private key,

• If i change private key, so public key will also change according to new private key.

Refresh Token

Now we are storing access token & refresh token both on client side cookie.

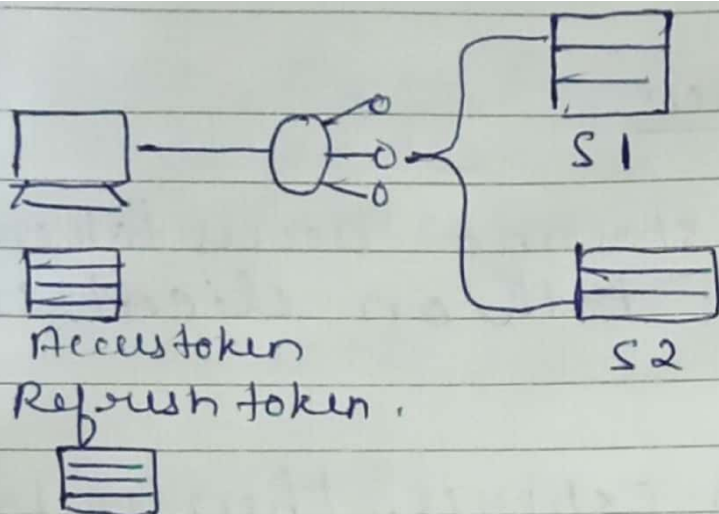
If access token expires, then refresh token refresh the token.

Note - In every request both Access & Refresh Token automatically send, but if access token expires then refresh token renew the access token.

We will create Refresh Token via using HS256 Algo.

Because here refresh token encrypt & decrypt via single key. And that key will store only on auth service.

We don't need to maintain that single key which is used to sign the refresh token.



HS256
Single key used
to encrypt &
decrypt the
token.

Why we need to store Refresh token in database, as we previously understood ki we have to create stateless Authentication system.

Token Rotation is an example that why we store Refresh token in dB.

Although we store Acces and Refresh token on client side, but for token rotation we need to store refresh token in dB.